

JUnit

Software Testing Lab

What is JUnit



- Junit is a Unit Testing framework for Java
- The framework is used by Java Developers to write and execute tests
- Every time a new code is added, all the test cases have to be re-executed

Unit Testing

UNIT TESTING

- Unit Testing refers to the testing of small chunks of codes
- It helps in early identification of defects
- The developers are tend to spend more time on reading the code

Two types of Unit Testing

Manual Testing

- If the test cases are tested manually without any automated tool, it is manual testing.
- It is less reliable and time-consuming

Automated Testing

- If the test cases are tested by tool support, it is automated testing.
- It is more reliable and faster

Junit Installation

Java / Technical Details / Java SE /
Java SE Development Kit 8 Downloads

Java SE Downloads

Java SE Subscriptions

Java SE Development Kit 8 Downloads

Thank you for downloading this release of the Java™ Platform, Standard Edition Development Kit (JDK™). The JDK is a development environment for building applications, applets, and components using the Java programming language.

The JDK includes tools useful for developing and testing programs written in the Java programming language and running on the Java platform.

Important Oracle JDK License Update

The Oracle JDK License has changed for releases starting April 16, 2019.

The new [Oracle Technology Network License Agreement for Oracle Java SE](#) is substantially different from prior Oracle JDK licenses. The new license permits certain uses, such as personal use and development use, at no cost -- but other uses authorized under prior Oracle JDK licenses may no longer be available. Please review the terms carefully before downloading and using this product. An FAQ is available [here](#).

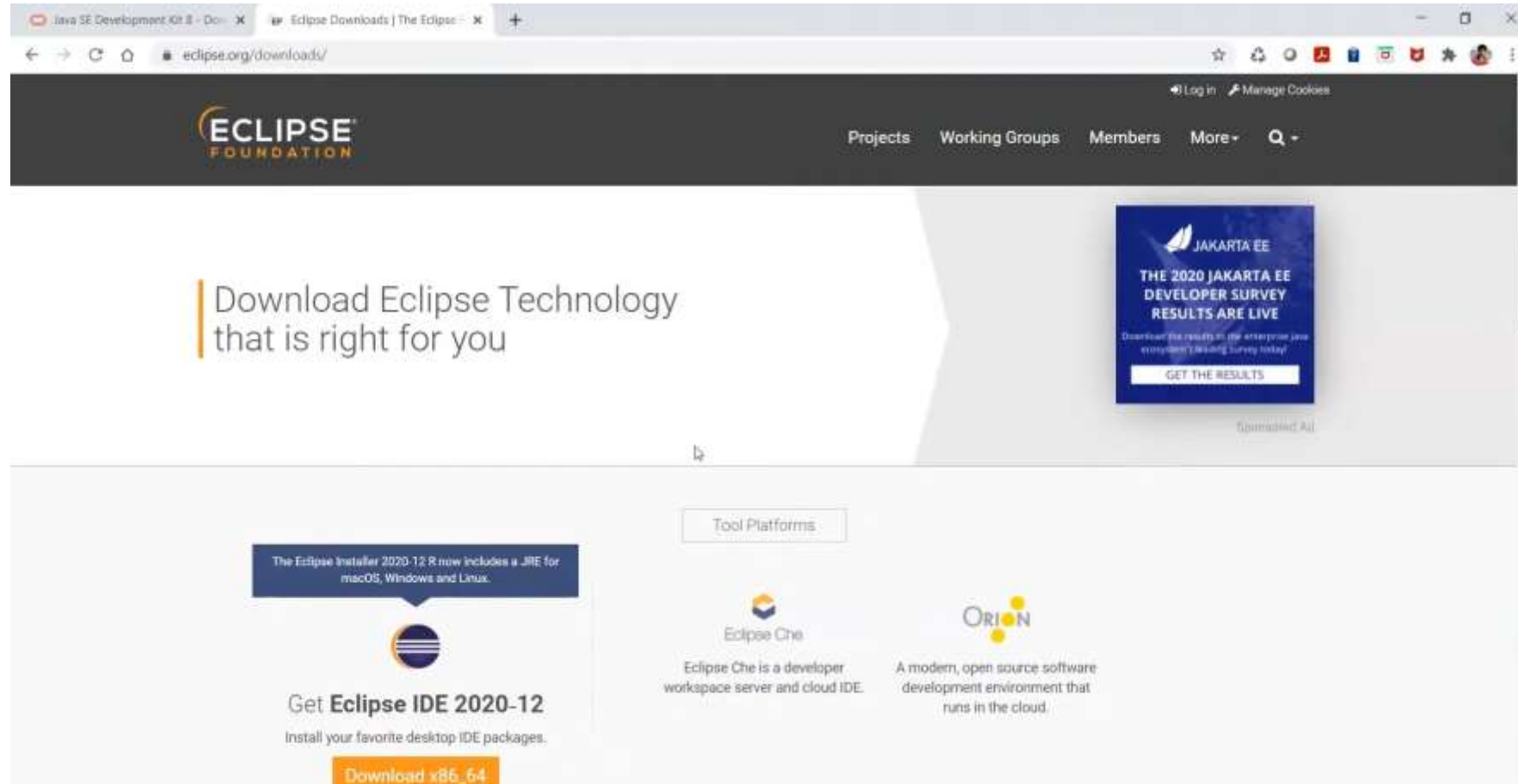
Commercial license and support is available with a low cost [Java SE Subscription](#).

Oracle also provides the latest OpenJDK release under the open source [GPL License](#) at [jdk.java.net](#).

For windows

Windows x86	154.48 MB	 jdk-8u271-windows-i586.exe
Windows x64	166.79 MB	 jdk-8u271-windows-x64.exe

Eclipse



The screenshot shows the Eclipse Foundation's download page. The browser's address bar displays 'eclipse.org/downloads/'. The page features a dark header with the Eclipse Foundation logo and navigation links for Projects, Working Groups, Members, and a search bar. The main content area has a large heading 'Download Eclipse Technology that is right for you'. To the right, there is a blue promotional box for the 'THE 2020 JAKARTA EE DEVELOPER SURVEY RESULTS ARE LIVE' with a 'GET THE RESULTS' button. Below this, a 'Tool Platforms' section highlights 'Eclipse Che' and 'ORION'. The 'Eclipse Che' section includes a callout box stating 'The Eclipse Installer 2020-12 R now includes a JRE for macOS, Windows and Linux.' and a 'Download x86_64' button. The 'ORION' section describes it as a modern, open source software development environment that runs in the cloud.

Log in Manage Cookies

ECLIPSE
FOUNDATION

Projects Working Groups Members More - Q -

Download Eclipse Technology
that is right for you

JAKARTA EE
**THE 2020 JAKARTA EE
DEVELOPER SURVEY
RESULTS ARE LIVE**
Download the results of the enterprise Java
ecosystem's leading survey today!
[GET THE RESULTS](#)

Sponsored Ad

Tool Platforms

The Eclipse Installer 2020-12 R now includes a JRE for macOS, Windows and Linux.


Get Eclipse IDE 2020-12
Install your favorite desktop IDE packages.
[Download x86_64](#)


Eclipse Che
Eclipse Che is a developer workspace server and cloud IDE.


ORION
A modern, open source software development environment that runs in the cloud.

Junit.org/junit4/

The screenshot shows the JUnit 5 website homepage. At the top, there's a navigation bar with 'JUnit 4' and 'Project Documentation'. Below it, a sub-header says 'JUnit 4: About' and 'Version: 4.13.1 Last Published: 2020-10-11'. The main content area features a large banner with the JUnit 5 logo (a red circle with a white '5' on a green background) and the text 'JUnit 5 is released: Explore the new release!'. Below the banner, there's a blue button that says 'Let's take a tour >'. The page is divided into three columns: 'Welcome', 'Usage and Idioms', and 'Third-party extensions'. The 'Welcome' column lists links for 'Download and install', 'Getting started', and 'Release Notes', with a sub-list of versions including 4.13.1, 4.13, 4.12, 4.11, 4.10, 4.9, and 4.9.1. The 'Usage and Idioms' column lists various testing concepts like Assertions, Test Runners, and Parameterized Tests. The 'Third-party extensions' column lists external libraries like Custom Runners and System Rules.

JUnit 4 • Project Documentation •

JUnit 4: About Version: 4.13.1 Last Published: 2020-10-11

JUnit is a simple framework to write repeatable tests. It is an instance of the xUnit architecture for unit testing frameworks.

JUnit 5

JUnit 5 is released:
Explore the new release!

Let's take a tour >

Welcome

- [Download and install](#)
- [Getting started](#)
- [Release Notes](#)
 - 4.13.1
 - 4.13
 - 4.12
 - 4.11
 - 4.10
 - 4.9.1
 - 4.9
- [Maintainer Documentation](#)

Usage and Idioms

- [Assertions](#)
- [Test Runners](#)
- [Aggregating tests in Suites](#)
- [Test Execution Order](#)
- [Exception Testing](#)
- [Matchers and assertThat](#)
- [Ignoring Tests](#)
- [Timeout for Tests](#)
- [Parameterized Tests](#)
- [Assumptions with Assume](#)
- [Rules](#)

Third-party extensions

- [Custom Runners](#)
- [net.trajano.commons:commons-testing-for-UtilityClassTestUtil](#) per #646
- [System Rules](#) – A collection of JUnit rules for testing code that uses java.lang.System.
- [JUnit Toolbox](#) - Provides runners for parallel testing, a [FoolingWait](#) class to ease asynchronous testing, and a [WildcardPatternSuite](#) which allow you to specify wildcard patterns instead of explicitly listing all classes when you create a suite

Welcome

- [Download and install](#)
- [Getting started](#)
- [Release Notes](#)
 - 4.13.1
 - 4.13
 - 4.12
 - 4.11
 - 4.10
 - 4.9.1
 - 4.9

JAR Files

The screenshot shows the GitHub Wiki page for JUnit 4. The page title is "Download and Install". Below the title, it says "Marc Philipp edited this page on Jan 1, 2020 · 34 revisions". The page content is divided into two main sections: "Plain-old JAR" and "Maven".

Plain-old JAR

To download and install JUnit you currently have the following options:

Download the following JARs and add them to your test classpath:

- `junit.jar`
- `hamcrest-core.jar`

Maven

Add a dependency to `junit:junit` in `test` scope. (Note: 4.13 is the latest stable version as of the latest edit on this page.)

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13</version>
  <scope>test</scope>
</dependency>
```

On the right side of the page, there is a sidebar with a search bar and a list of pages. The list includes: Home, 'Enclosed' test runner example, Aggregating tests in suites, Assertions, Assertions CN, Assumptions with assume, Categories, Continuous testing, Custom runners, Deprecation Policy, and Developing with fast tests.

Needed JAR Files

Download the following JARs and add them to your test classpath:

- `junit.jar`
- `hamcrest-core.jar`

org.hamcrest:hamcrest-core:1.3



org.hamcrest:hamcrest-core: 1.3

View on OSS Index

Browse

Download

Hamcrest Core

This is the core API of hamcrest matcher framework to be used by third-party framework providers. This includes the a foundation set of matcher implementations for common operations.



Apache Maven

maven.apache.org

```
<dependency>
  <groupId>org.hamcrest</groupId>
  <artifactId>hamcrest-core</artifactId>
  <version>1.3</version>
</dependency>
```

jar
javadoc.jar
pom
sources.jar



org.hamcrest:hamcrest-core

1.3

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.hamcrest</groupId>
    <artifactId>hamcrest-parent</artifactId>
    <version>1.3</version>
  </parent>
```

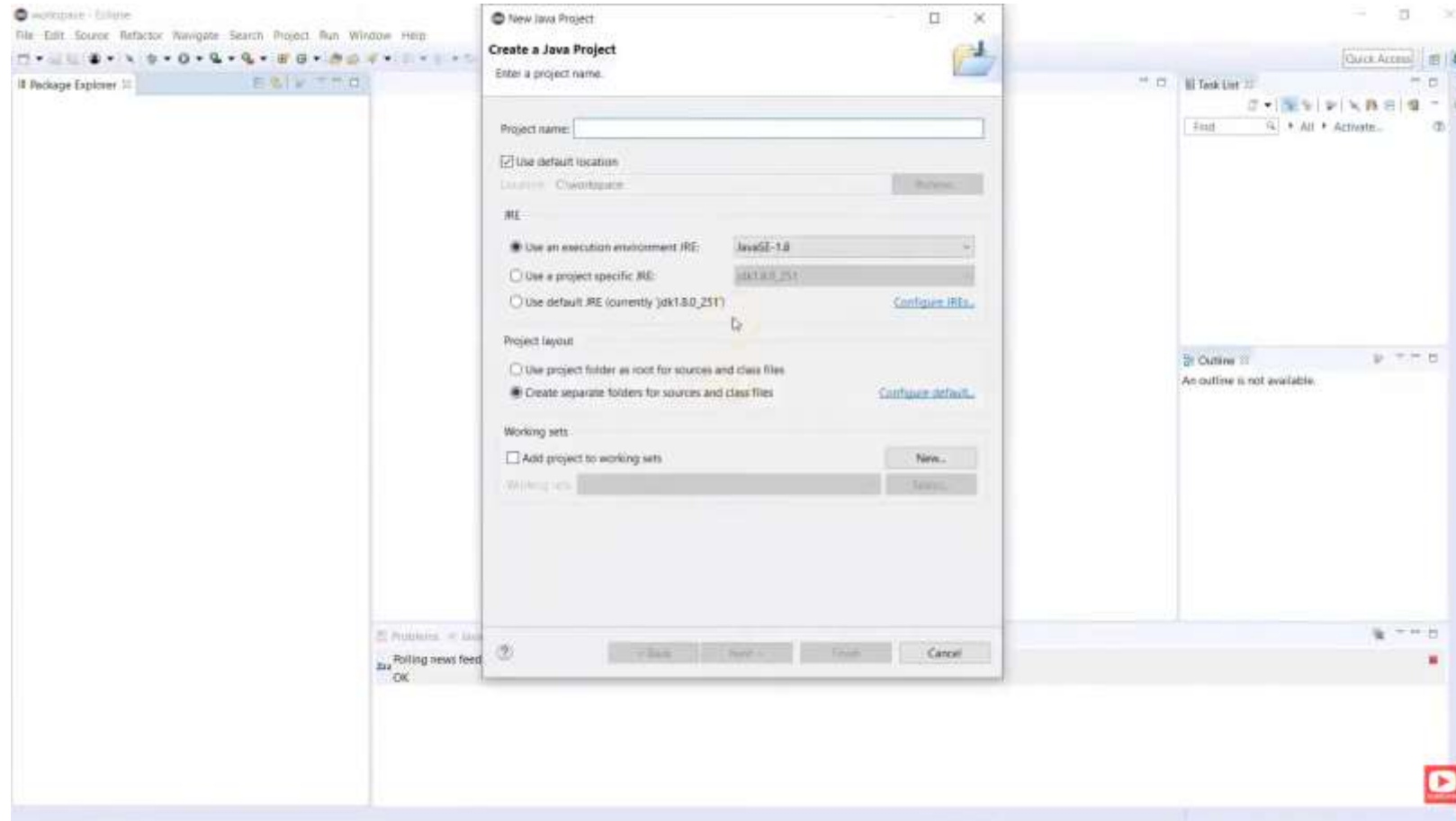


Gradle Groovy DSL

gradle.org

```
implementation 'org.hamcrest:hamcrest-core:1.3'
```

Eclipse New project Creation



Project name:

☒ Use default location

Location:

JRE

☒ Use an execution environment JRE:

☐ Use a project specific JRE:

☐ Use default JRE (currently 'jdk1.8.0_251') [Configure JREs...](#)

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets

Working sets:

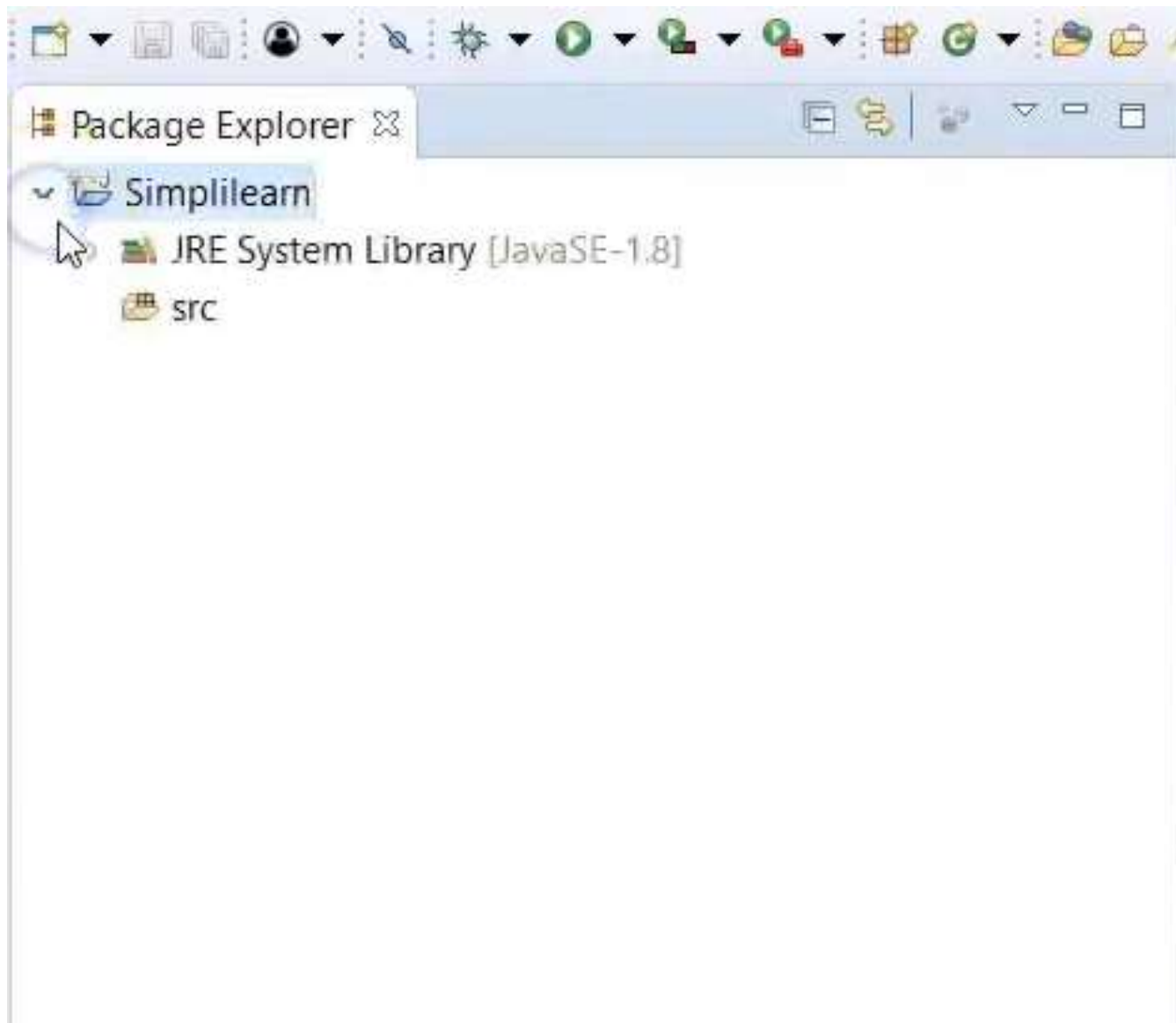


< Back

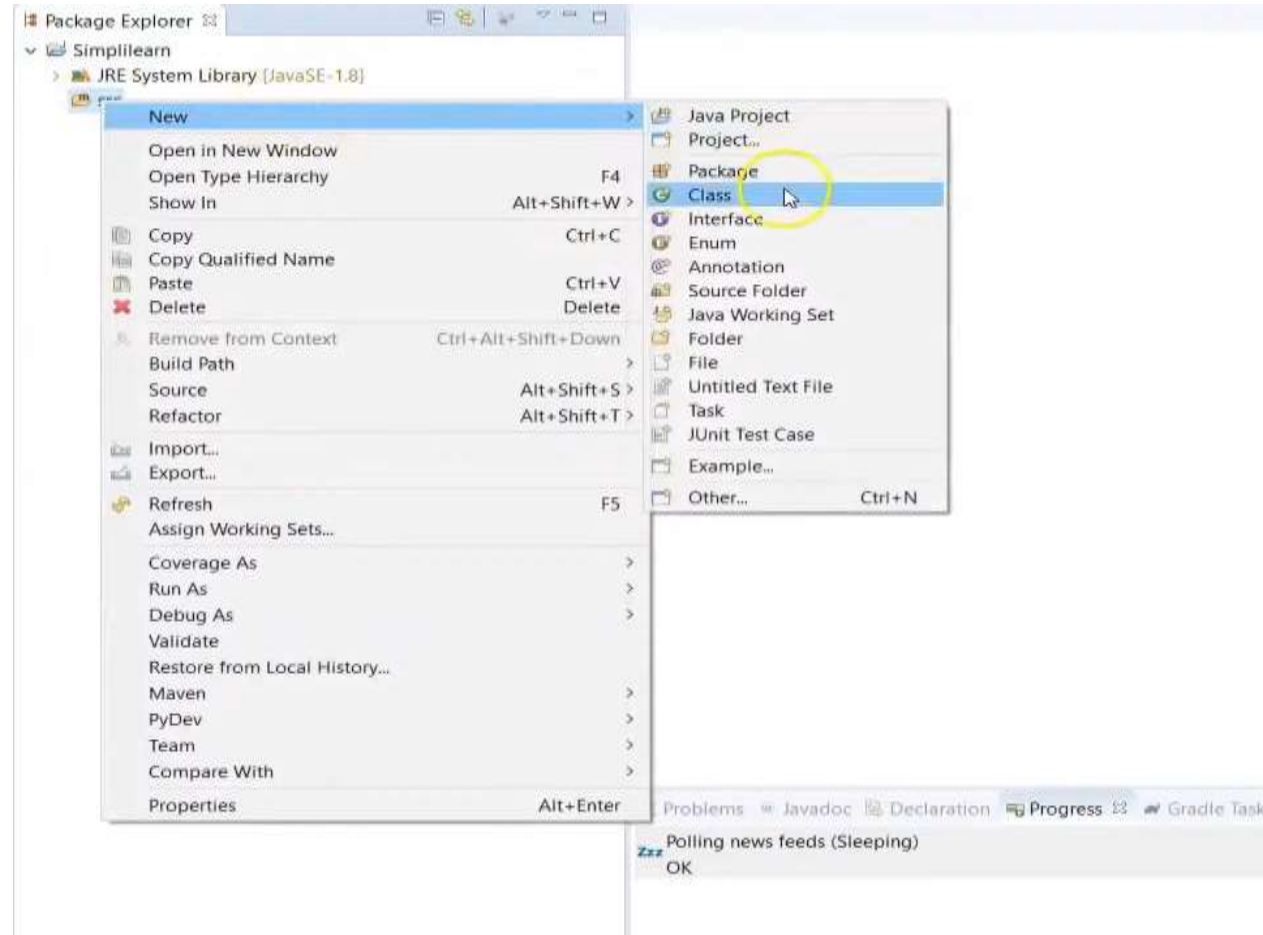
Next >

Finish

Cancel



Create a New Class



Give Class Name

New Java Class

Java Class

The use of the default package is discouraged.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass:

Interfaces:

Which method stubs would you like to create?

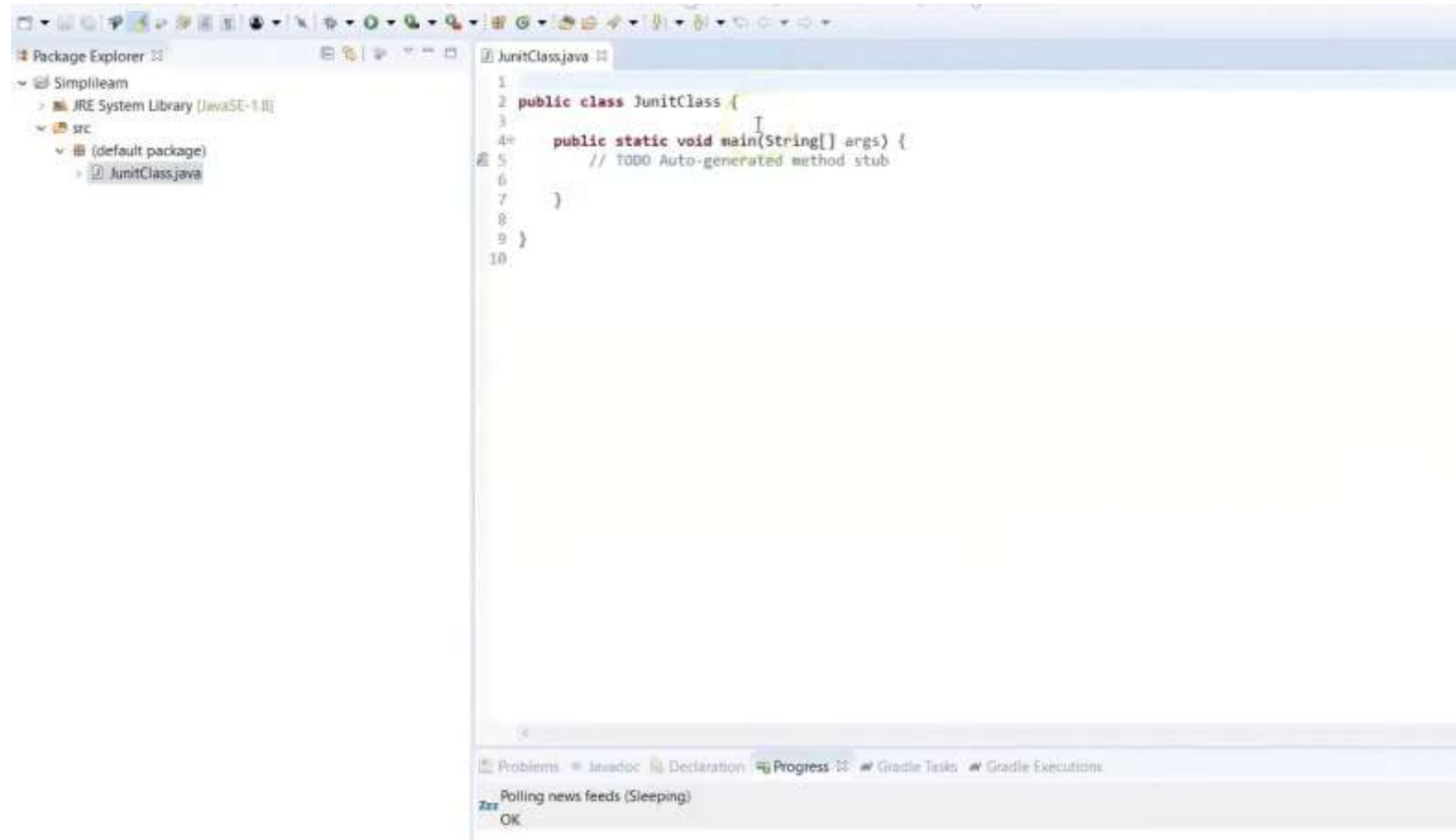
☒ public static void main(String[] args)

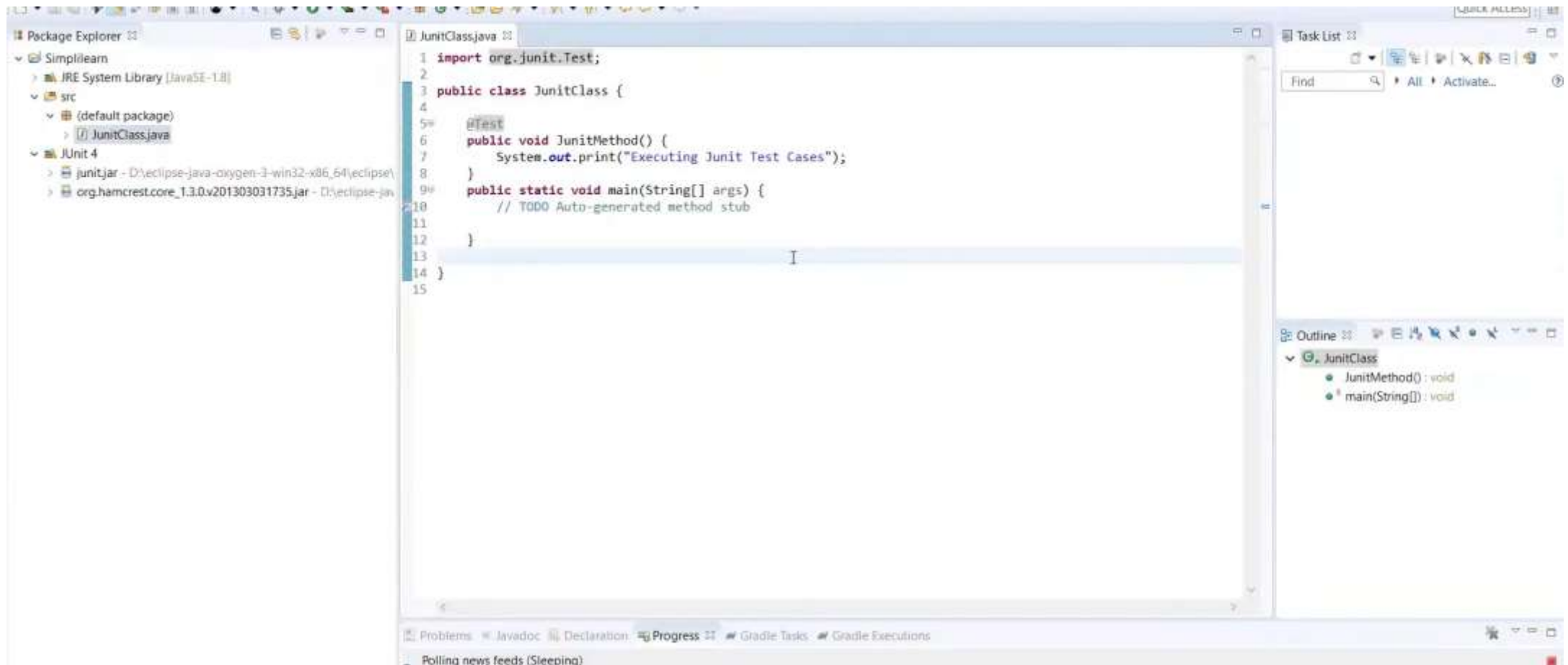
☐ Constructors from superclass

☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments





JunitClass.java

```
1 import org.junit.Test;
2
3 public class JunitClass {
4
5     @Test
6     public void JunitMethod() {
7         System.out.print("Executing Junit Test Cases");
8     }
9     public static void main(String[] args) {
10         // TODO Auto-generated method stub
11
12     }
13
14 }
15
```

Features of JUnit





JUnit provides assertions to test expected results



Provides Assertions

The tests in JUnit are automatically run, that provide instant feedback



Automated Test Running

Open Source Network



JUnit is an open source network used to write and run tests

JUnit provides test runners to run the tests

Provides Test Runners



JUnit allows faster code writing, that increases the code's quality

Improved Code Quality



JUnit provides annotations to identify test methods

Provides Annotations



JUnit Annotations

- JUnit Annotations refer to the syntactic meta-data added to the Java source code for better structure and readability



Some important and frequently used JUnit Annotations



@Test

Tells JUnit which public void method can be run as a test case

@Before

To execute some statement before each test case

@After

To execute some statement after each test case

@Ignore

Used to ignore some statements during test execution



Some important and frequently used JUnit Annotations



@BeforeClass

Used to execute a statement before all the test cases

@AfterClass

Used to execute a statement after all the test cases

@Test
(time out =500)

Used to set some timeout while executing the test

@Test
(expected=Illegal
ArgumentException.class)

Used to handle some exception during test execution



- **@Test**
- **Purpose**
- Marks a method as a test case.
- JUnit will execute only methods annotated with @Test.
- **Use**
- To tell JUnit:
- “This method should run as a test.”

Example:

- `import org.junit.Test;`
- `import static org.junit.Assert.*;`
- `public class CalculatorTest {`
- `@Test`
- `public void testAddition() {`
- `int result = 2 + 3;`
- `assertEquals(5, result);`
- `}`
- `}`

- **@Test** → “This is a test method”
- Purpose:
- Tells JUnit: "Run this method as a test."
- Example:
- @Test
- public void testAddition() {
- int result = 2 + 3;
- assertEquals(5, result);
- }
- If 5 is correct → test passes
If wrong → test fails

Simple meaning:

- This method is a test.

- **@Before → “Run this before every test”**
- Purpose:
Used to set up something before each test.
- Example:

```

public class ShoppingCartTest {
    private ShoppingCart cart;

    @Before
    public void setup() {
        cart = new ShoppingCart(); // fresh cart before each test    System.out.println("Setup before test");
    }

    @Test
    public void testAddItem() {
        cart.addItem("Book");
        assertEquals(1, cart.getItemCount());
    }

    @Test
    public void testRemoveItem() {
        cart.addItem("Book");
        cart.removeItem("Book");
        assertEquals(0, cart.getItemCount());
    }

    @Test
    public void testCheckoutEmptyCart() {
        assertFalse(cart.checkout());
    }
}

```

- **Simple meaning:** @Before = Reset or prepare things before every test runs, so tests don't interfere with each other.

- **@After → “Run this after every test”**
- Purpose:
Clean up after each test.
- Example:
- @After
- public void cleanup() {
- System.out.println("Cleanup after test");
- }

- **@BeforeClass** → “Run once before all tests”
- Purpose:
Runs only one time at the beginning.
- Must be static.
- Example:
- @BeforeClass
- public static void start() {
- System.out.println("Runs once");
- }
- Simple meaning:
Do something once before all tests.

- **@AfterClass** → “Run once after all tests”
- Purpose:
Runs only one time at the end.
- Example:
- @AfterClass
- public static void end() {
- System.out.println("Runs once at end");
- }
- Simple meaning:
Finish everything at end.

@Ignore → “Skip this test”

Purpose: Do not run this test.

Example:

```
@Ignore
```

```
@Test
```

```
public void testSomething() {  
}
```

Simple meaning:

Skip this test.

- **@Test(timeout = 500)**
- Purpose:
Fail test if it runs too long.
- Example:
- @Test(timeout = 500)
- public void testSpeed() throws Exception {
- Thread.sleep(600);
- }
- This will FAIL (too slow).
- Simple meaning:
Test must finish in time.

- **@Test(expected = Exception.class)**
- Purpose:
Pass test only if exception happens.
- Example:
- @Test(expected = ArithmeticException.class)
- public void testDivideByZero() {
- int x = 5 / 0;
- }
- If exception happens → PASS
If not → FAIL
- Simple meaning:
Expect error.

Annotation

What It Means (Simple)

@Test

This is a test

@Before

Run before each test

@After

Run after each test

@BeforeClass

Run once at start

@AfterClass

Run once at end

@Ignore

Skip this test

timeout

Fail if too slow

expected

Pass if exception occurs

Quick access

OneDrive

This PC

3D Objects

Desktop

Documents

Downloads

Music

Pictures

Videos

Windows-SSD (C:)

Data (D:)

New Volume (E:)

Network

Name	Date modified	Type	Size
hamcrest-core-1.3	02-01-2021 11:40 AM	Executable Jar File	44 KB
junit-4.13.1	02-01-2021 11:39 AM	Executable Jar File	374 KB
TestJunit.java	02-01-2021 12:07 PM	JAVA File	1 KB

TestJunit.java - Notepad

File Edit Format View Help

```
import org.junit.Test;
import org.junit.Ignore;
import static org.junit.Assert.assertEquals;

public class TestJunit {

    String message = "Simplilearn";

    @Test
    public void testPrintMessage() {
        System.out.println("Inside testPrintMessage()");
        assertEquals(message, "Simplilearn");
    }
    public static void main(String[] args) {
        TestJunit result = new TestJunit();
        result.testPrintMessage();
    }
}
```


C:\Windows\System32\cmd.exe

Microsoft Windows [Version 10.0.19042.685]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\Desktop\junit4>javac -cp junit-4.13.1.jar;. TestJUnit.java

C:\Users\Lenovo\Desktop\junit4>java -cp junit-4.13.1.jar;. TestJUnit
Inside testPrintMessage()

C:\Users\Lenovo\Desktop\junit4>

```
C:\Users\Lenovo\Desktop\junit4>javac -cp junit-4.13.1.jar;. TestJUnit.java
```

```
C:\Users\Lenovo\Desktop\junit4>java -cp junit-4.13.1.jar;. TestJUnit
```

```
Inside testPrintMessage()
```

```
Exception in thread "main" org.junit.ComparisonFailure: expected:<Simplilearn[]> but was:<Simplilearn[12]>
```

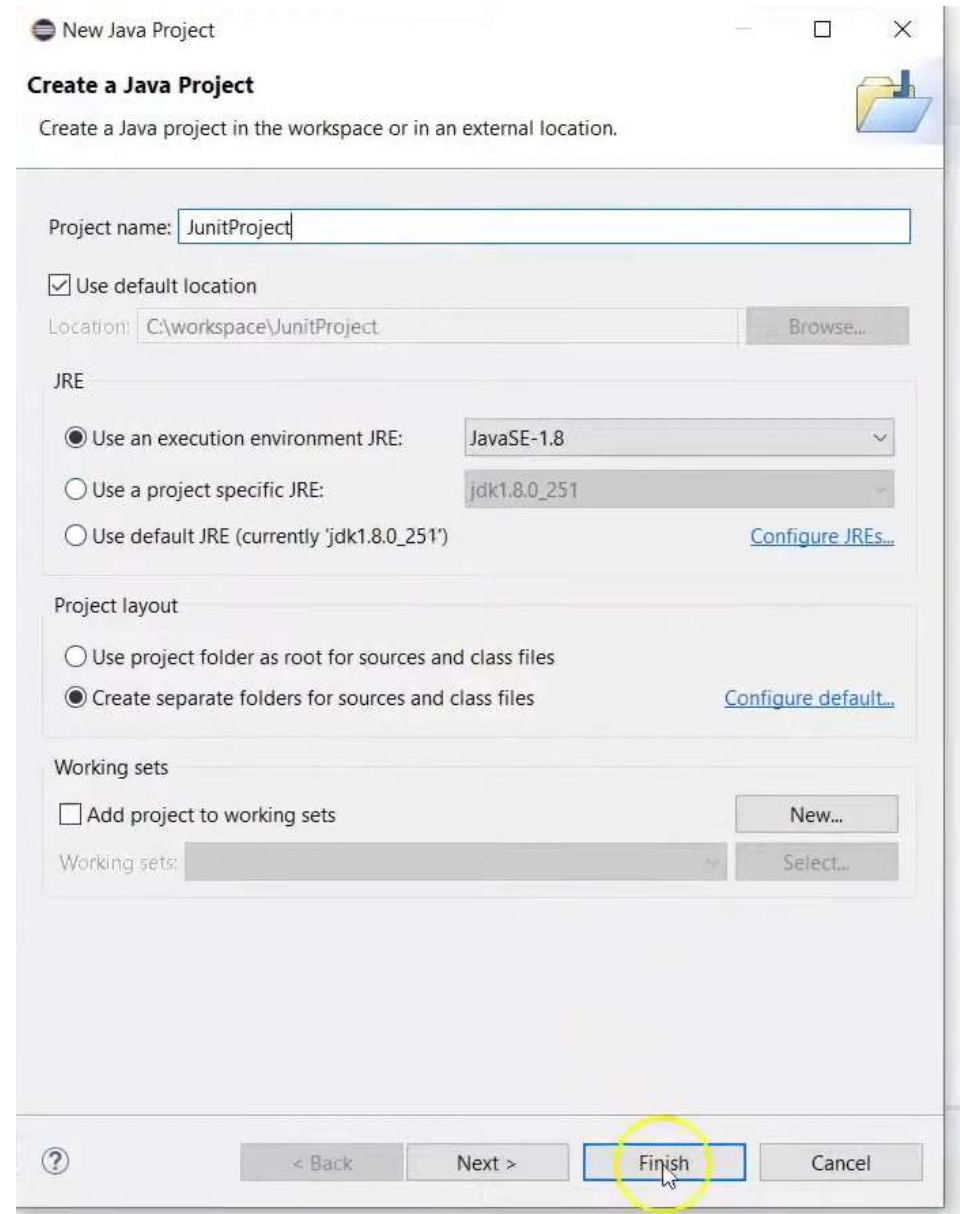
```
    at org.junit.Assert.assertEquals(Assert.java:117)
```

```
    at org.junit.Assert.assertEquals(Assert.java:146)
```

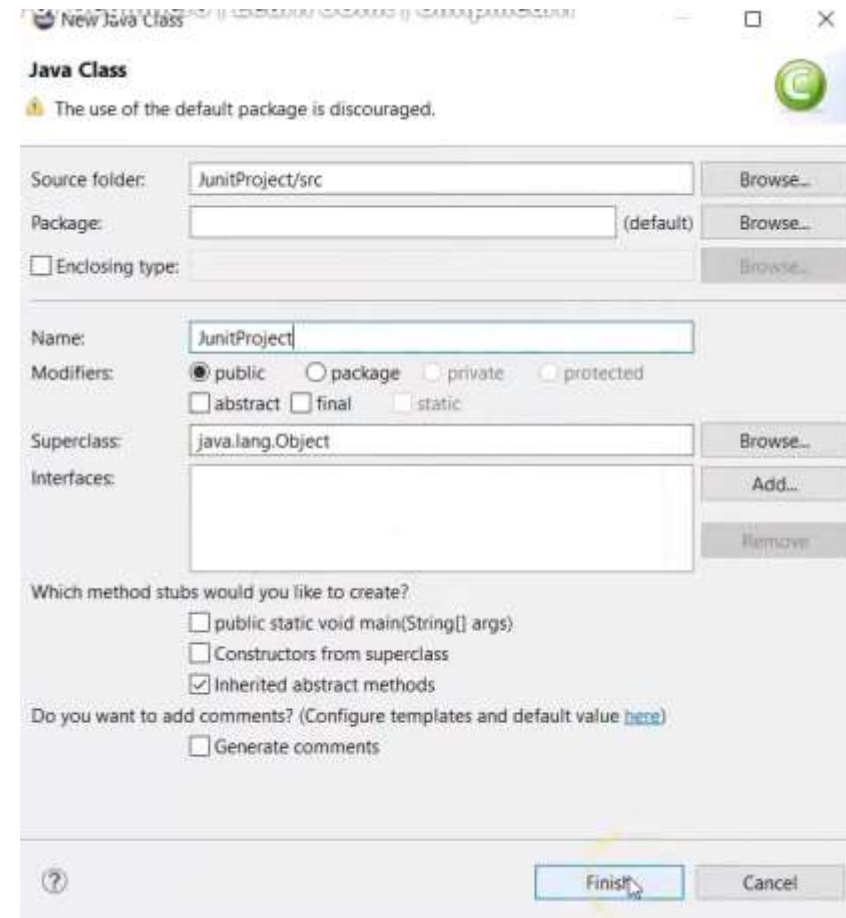
```
    at TestJUnit.testPrintMessage(TestJUnit.java:12)
```

```
    at TestJUnit.main(TestJUnit.java:16)
```

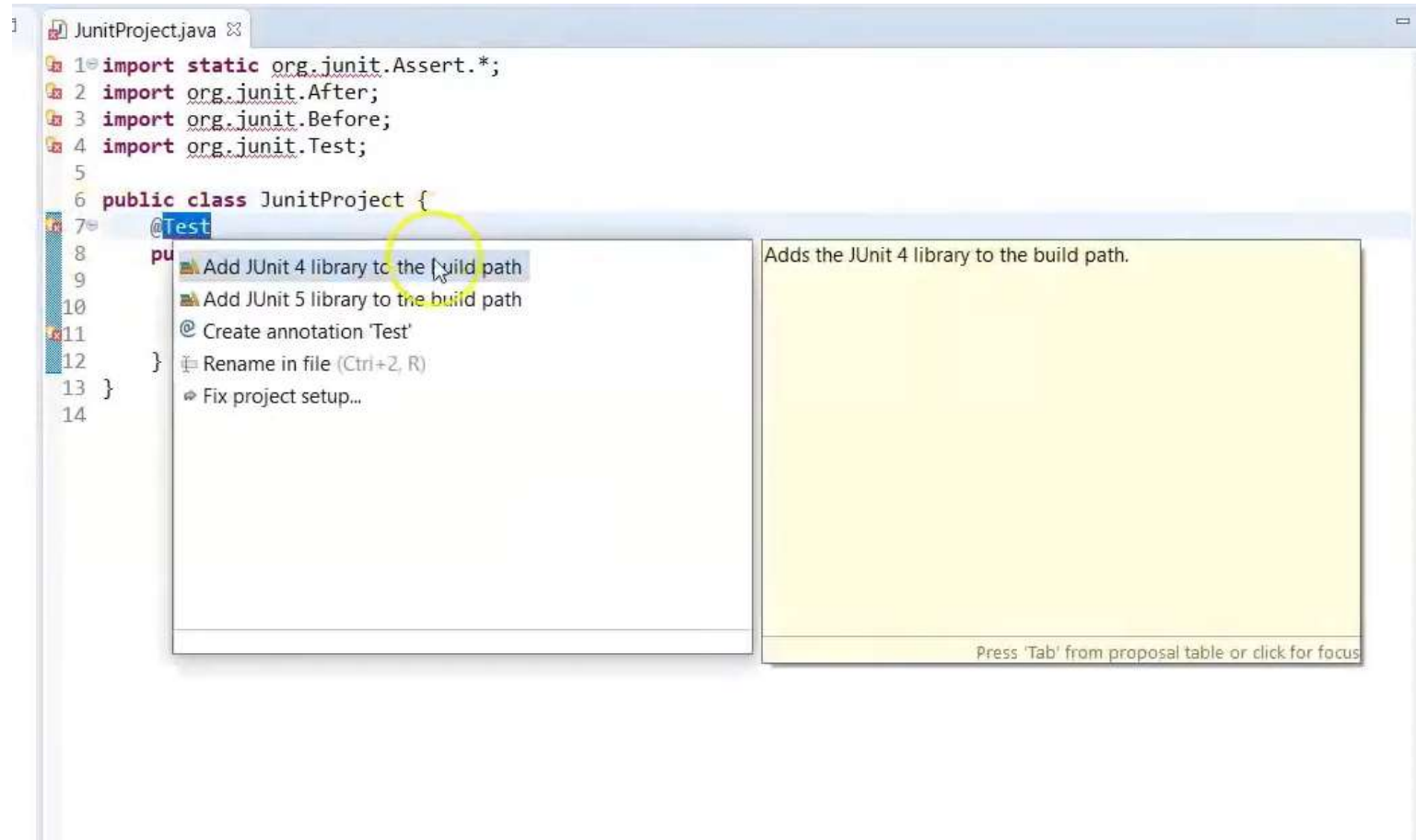
JUnit Testing in Eclipse



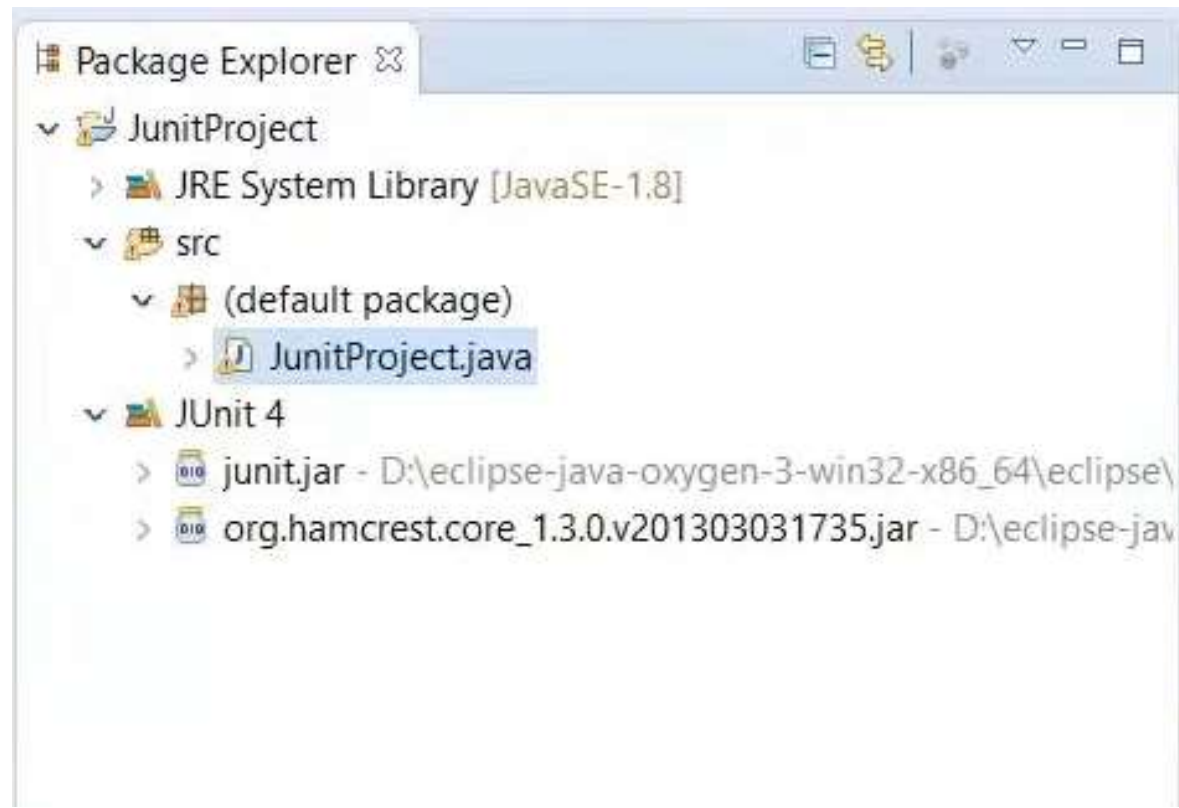
Create Class File (right click on src)



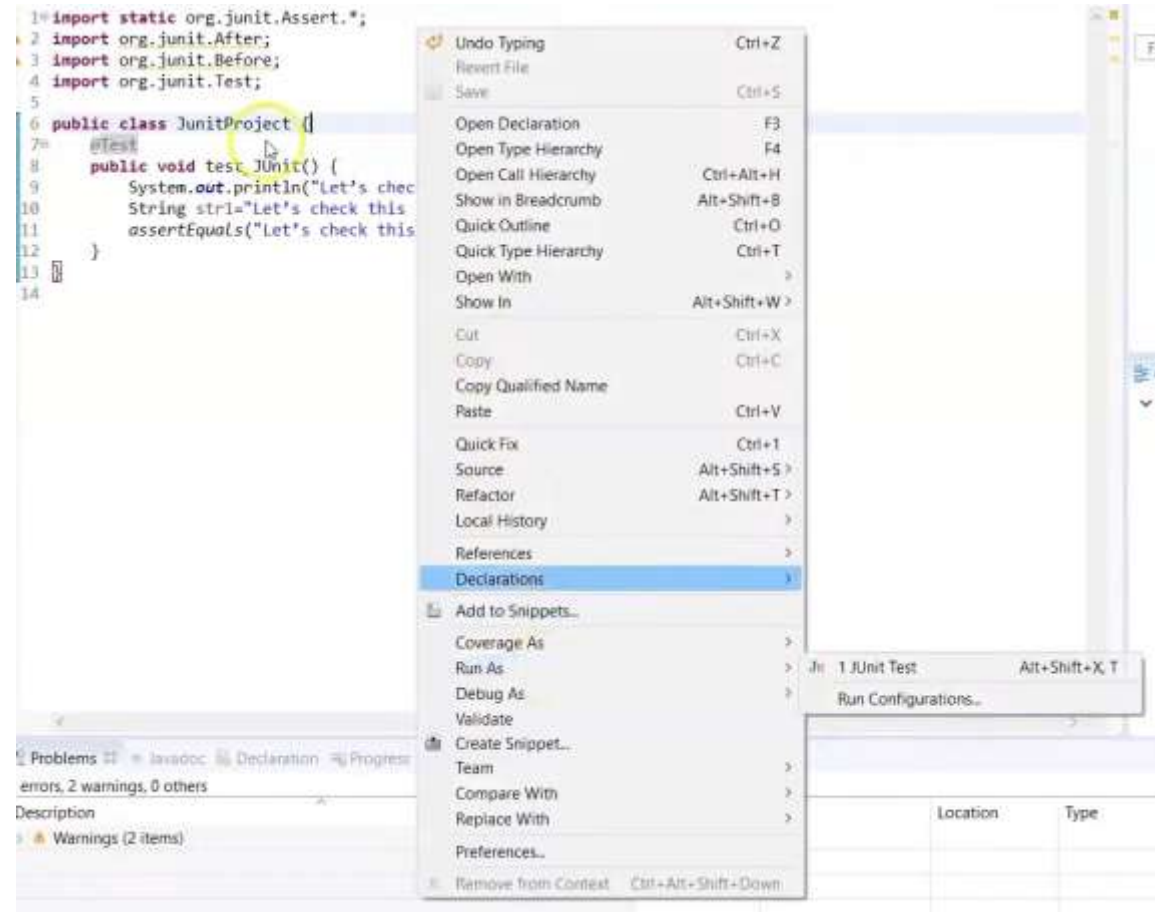
Add Junit 4 library

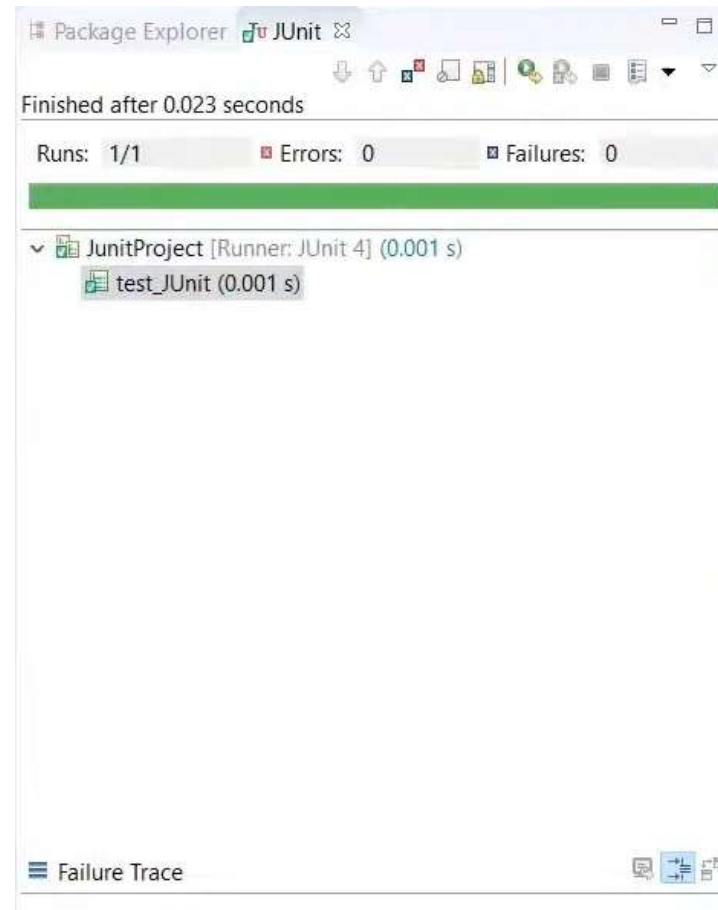


After adding junit (you can see Like this)



Right Click- Run As – Junit Test





workspace - JUnitProject/src/JUnitProject.java - Eclipse

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer JUnit

JUnitProject.java

```
1 import static org.junit.Assert.*;
2 import org.junit.After;
3 import org.junit.Before;
4 import org.junit.Test;
5
6 public class JUnitProject {
7     @Test
8     public void test_JUnit() {
9         System.out.println("Let's check this string");
10        String str1="Let's check this string1";
11        assertEquals("Let's check this string", str1);
12    }
13 }
14
```

Task List

Find

Outline

- JUnitProject
 - test_JUnit() : void

Failure Trace

org.junit.ComparisonFailure: expected:<...'s check this string[]> but was:<...'s check this string[1]>
at JUnitProject.test_JUnit(JUnitProject.java:11)

Java Stack Trace Console

org.junit.ComparisonFailure: expected:<...'s check this string[]> but was:<...'s check this string[1]>
at org.junit.Assert.assertEquals(Assert.java:115)
at org.junit.Assert.assertEquals(Assert.java:144)
at JUnitProject.test_JUnit(JUnitProject.java:11)
at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
at java.lang.reflect.Method.invoke(Method.java:498)

Writable Smart Insert 10:46

